



SIMD-Intrinsics

SIMD Programming

(CS 3006)

Dr. Muhammad Arshad Islam

Based on Material by M. Püschel, ETH Zürich

Department of Computer Science,
National University of Computer & Emerging Sciences,
Islamabad Campus



Recap-Parallel Architectures

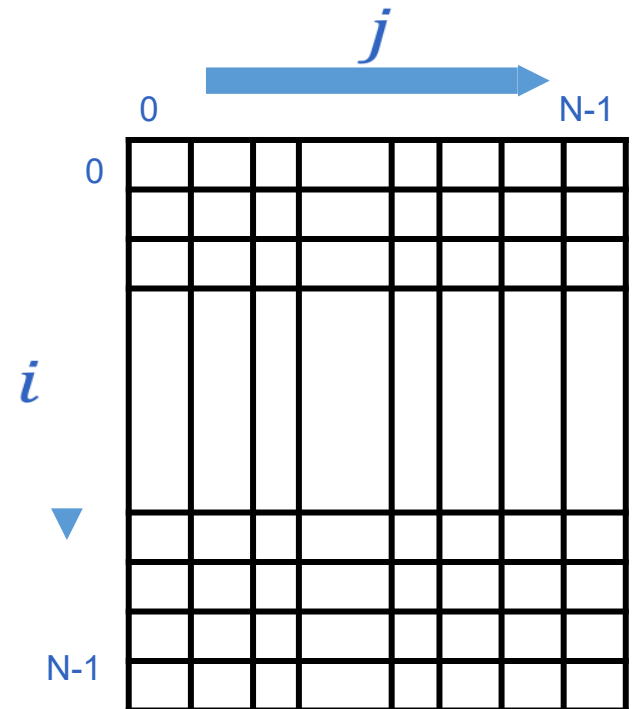
		Instruction Streams	
		one	many
Data Streams	one	SISD traditional von Neumann single CPU computer	MISD May be pipelined Computers
	many	SIMD Vector processors fine grained data Parallel computers	MIMD Multi computers Multiprocessors



2D-Matrices

- Square matrix of dimension $N \times N$
 - i indexes through rows
 - j indexes through columns

3

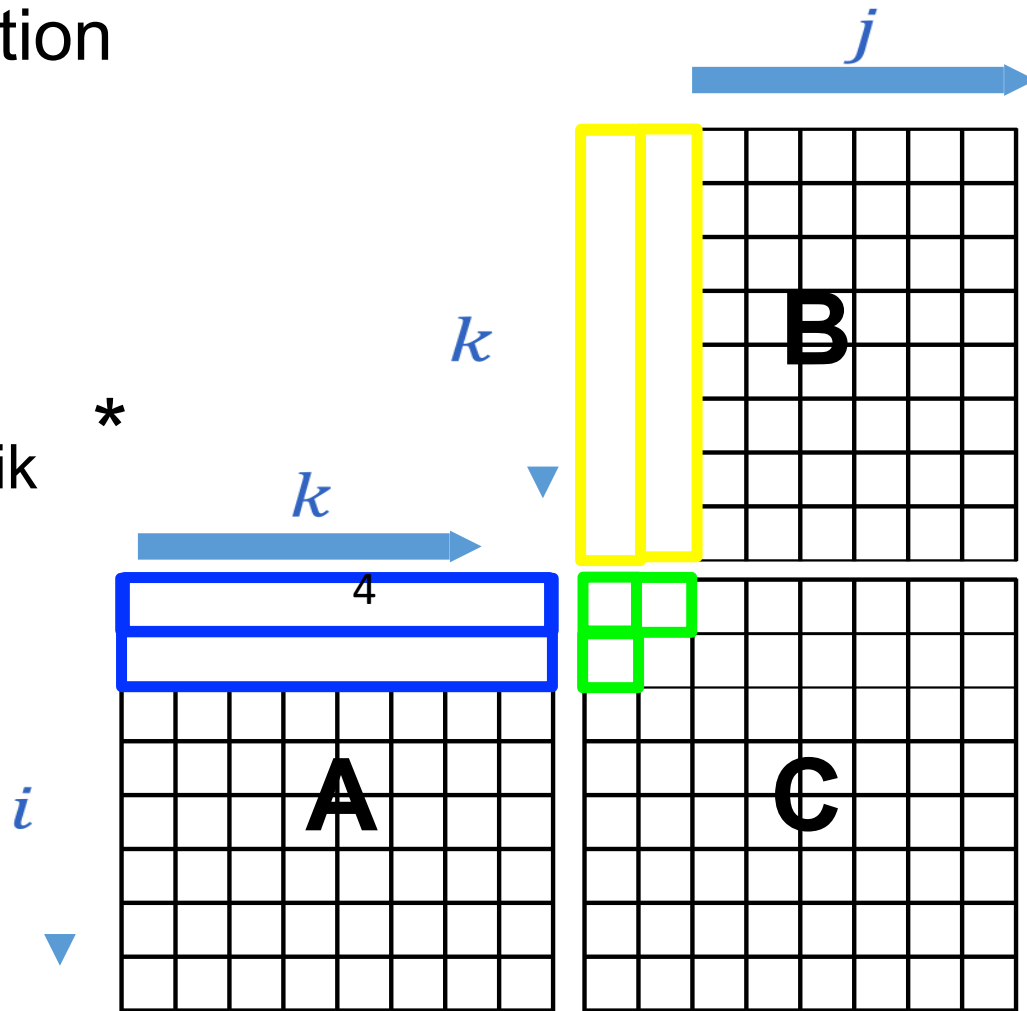




Matrix Multiplication

$$\mathbf{C} = \mathbf{A} * \mathbf{B}$$

$$C_{ij} = \sum_k (A_{ik} * B_{kj})$$





2D Matrix Memory Layout

- $a[][]$ in C uses row-major
- Fortran uses column-major
- Our examples use column-major

a_i

j	a_{00}	a_{01}	a_{02}	a_{03}
	a_{10}	a_{11}	a_{12}	a_{13}
	a_{20}	a_{21}	a_{22}	a_{23}
	a_{30}	a_{31}	a_{32}	a_{33}

5

Row-Major

	a_{13}	
	a_{12}	
	a_{11}	
Row	a_{10}	
	a_{03}	
	a_{02}	
	a_{01}	
Row	a_{00}	

Column-Major

	a_{31}	
	a_{21}	
	a_{11}	
	a_{01}	
	a_{30}	
	a_{20}	
	a_{10}	
Column	a_{00}	

$$a_{ij} : a[i * N + j]$$

$$a_{ij} : a[i + j * N]$$



Simplifying Assumptions...

- We want to keep the examples (somewhat) manageable...
- We will keep the matrixes square
 - So both matrixes are the same size
with the same number of rows and columns
- We will keep the matrixes reasonably aligned
 - So size % a reasonable power of 2 == 0
- We are doing **dense** matrix multiplication
 - A related problem is "sparse" matrix multiplication, where most of the entries are 0



dgemm Reference Code: Python

```
def dgemm(N, a, b, c):  
    for i in range(N):  
        for j in range(N):  
            c[i+j*N] = 0  
            for k in range(N):  
                c[i+j*N] += a[i+k*N] * b[k+j*N]
```

N	Python [Mflops]
32	5.4
160	5.5
480	5.4
960	5.3

- 1 MFLOP = 1 Million floating-point operations per second (fadd, fmul)
- **dgemm**(N ...) takes $2 \cdot N^3$ flops



Timing Program Execution

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    // start time
    // Note: clock() measures execution time, not real time
    //       big difference in shared computer environments
    //       and with heavy system load
    clock_t start = clock();

    // task to time goes here:
    // dgemm(N, ...);

    // "stop" the timer
    clock_t end = clock();

    // compute execution time in seconds
    double delta_time = (double)(end-start)/CLOCKS_PER_SEC;
}
```

8

```
double Gflops = 2e-9*mat_size*mat_size*mat_size/delta_time;
```



C versus Python

N	C [GFLOPS]	Python [GFLOPS]
32	1.30	0.0054
160	1.30	0.0055
480	1.32	0.0054
960	0.91	0.0053



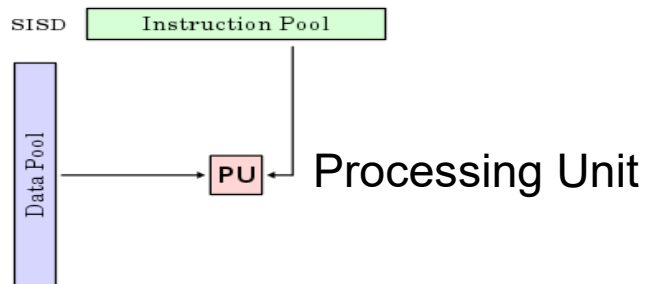
9

Which other class gives you this kind of power? We could stop here ... but why? Let's do better!



Single-Instruction/Single-Data Stream (SISD)

- Sequential computer that exploits no parallelism in either the instruction or data streams. Examples of SISD architecture are traditional uniprocessor machines
 - E.g. Our RISC-V processor
 - We consider superscalar as SISD because the **programming model** is sequential

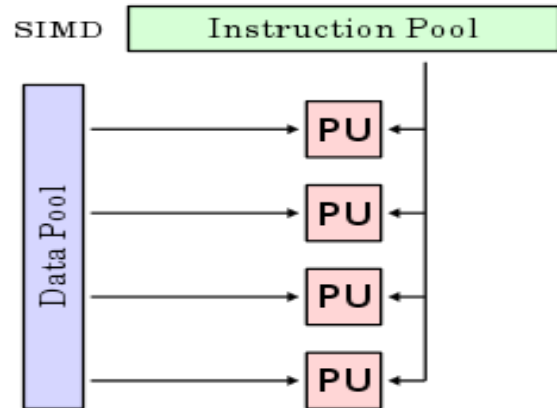


1
0



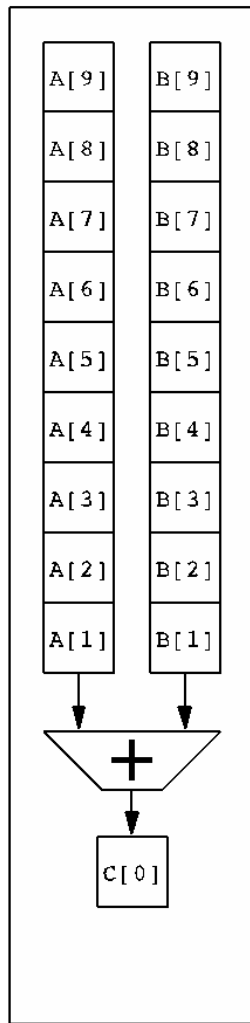
Single-Instruction/Multiple-Data Stream (SIMD or “sim-dee”)

- SIMD computer processes multiple data streams using a single instruction stream, e.g., Intel SIMD instruction extensions or NVIDIA Graphics Processing Unit (GPU)

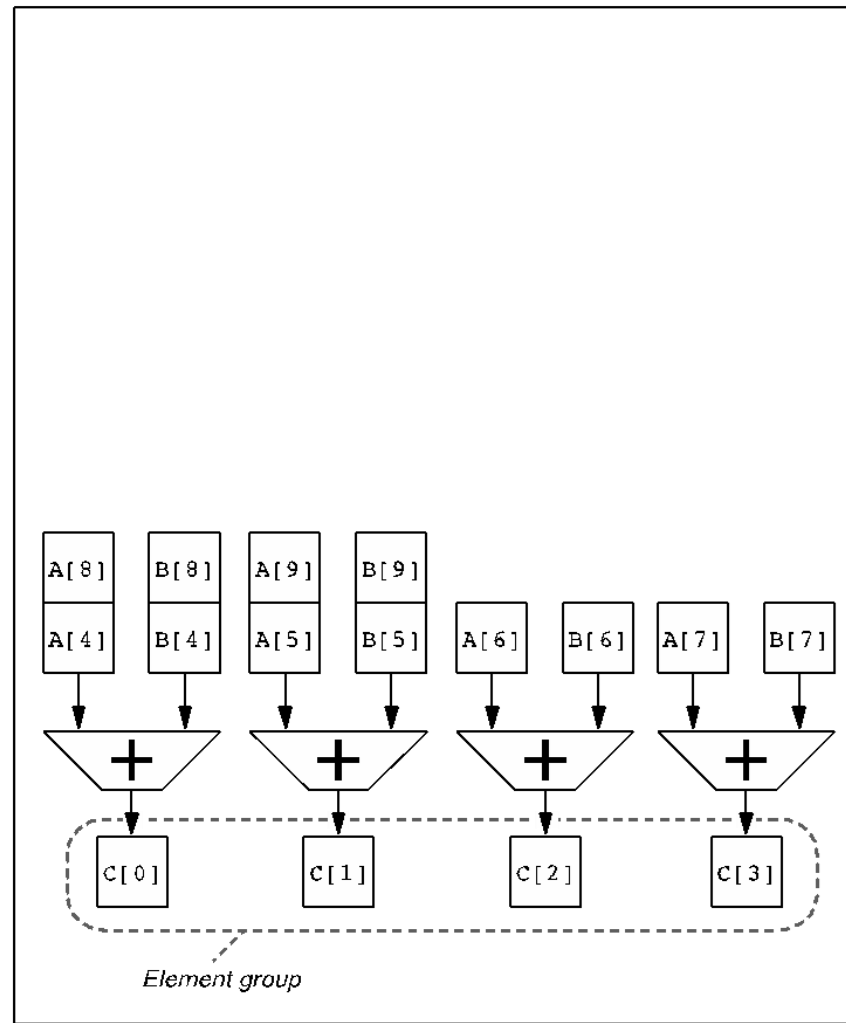


1
1

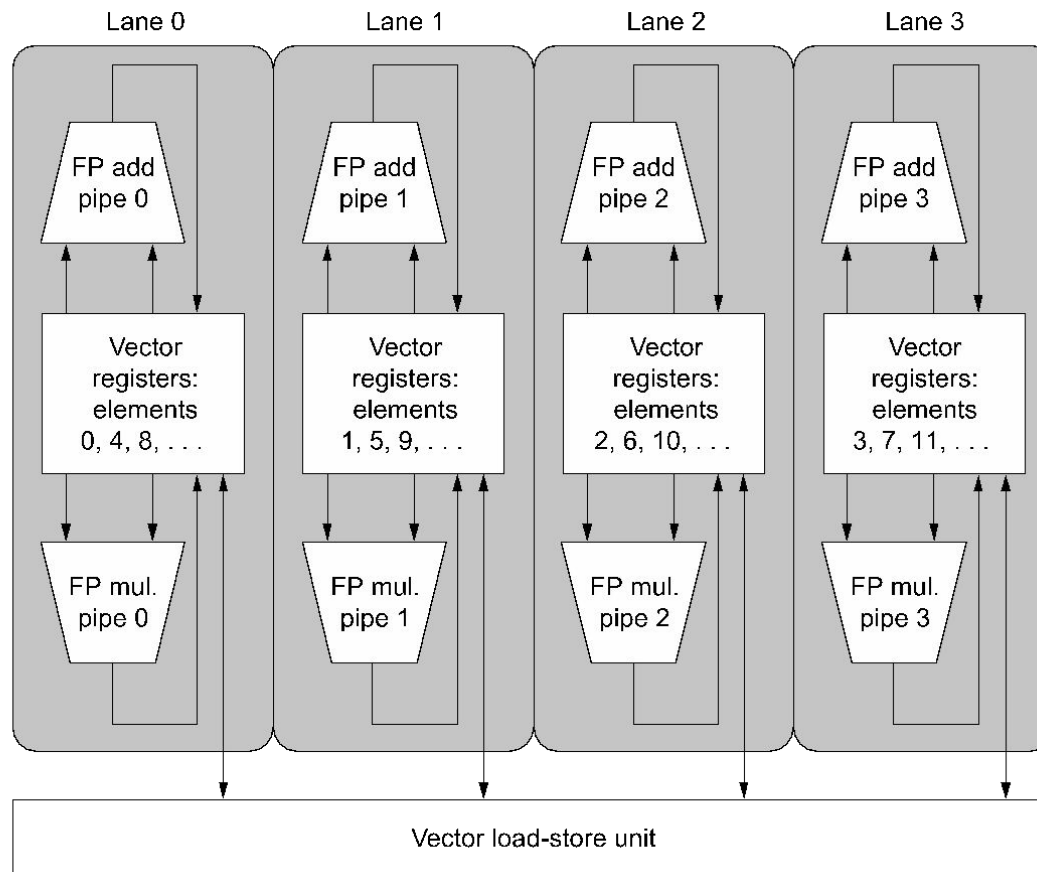
Today's topic.



(A)



(B)



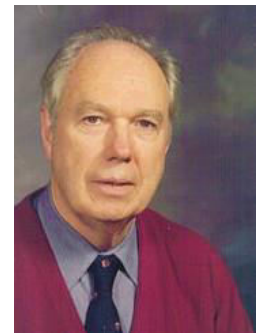


Flynn* Taxonomy, 1966

		Data Streams	
		Single	Multiple
Instruction Streams	Single	SISD: Intel Pentium 4	SIMD: SSE instructions of x86
	Multiple	MISD: No examples today	MIMD: Intel Xeon e5345 (Clovertown)

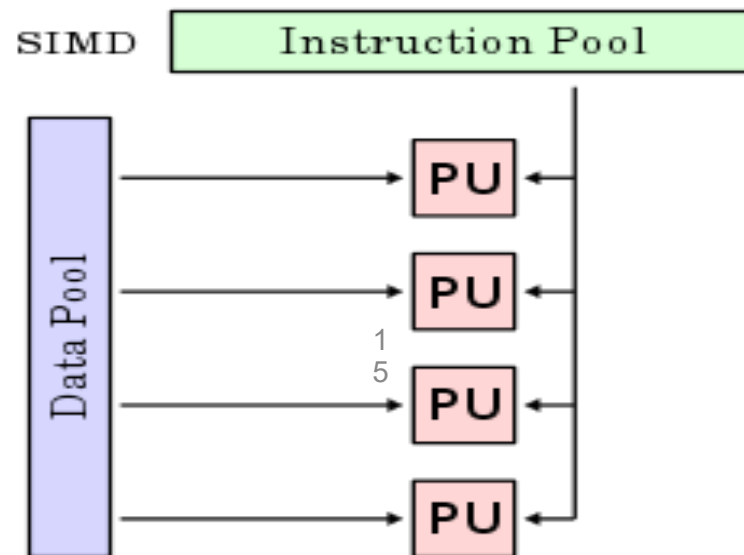
- SIMD and MIMD are currently the most common parallelism in architectures – usually both in same system!
- Most common parallel processing programming style: Single Program Multiple Data (“SPMD”)
 - Single program that runs on all processors of a MIMD
 - Cross-processor execution coordination using synchronization primitives

*Prof.
Michael
Flynn,
Stanford





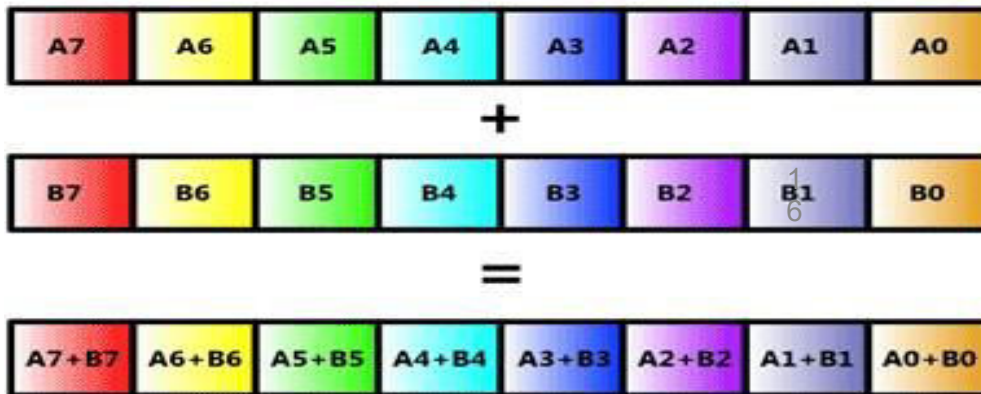
SIMD – “Single Instruction Multiple Data”



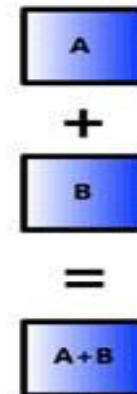


SIMD (Vector) Mode

SIMD Mode



Scalar Mode





SIMD Applications & Implementations

- Applications

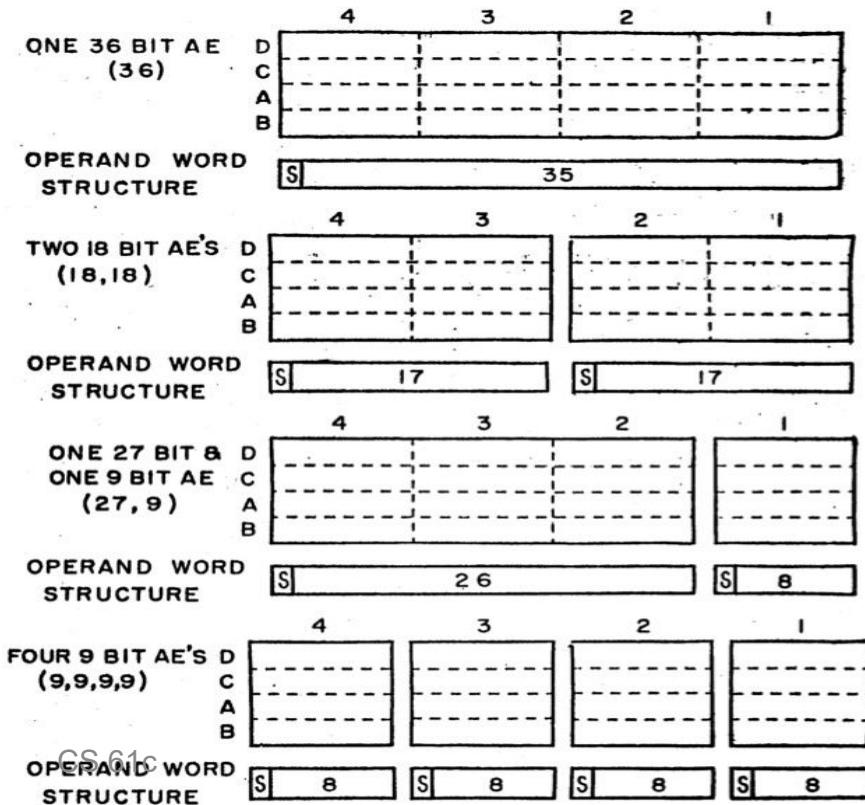
- Scientific computing
 - Matlab, NumPy
- Graphics and video processing
 - Photoshop, ...
- Big Data
 - Deep learning
- Gaming

- Implementations

- x86
- ARM
- RISC-V vector extensions
- Video cards
-



First SIMD Extensions: MIT Lincoln Labs TX-2, 1957



18



Intel x86 SIMD: Continuous Evolution

MMX 1997

1999	2000	2004	2006	2007	2008	2009	2010\11
SSE	SSE2	SSE3	SSSE3	SSE4.1	SSE4.2	AES-NI	AVX
70 instr Single-Precision Vectors Streaming operations	144 instr Double-precision Vectors 8/16/32 64/128-bit vector integer	13 instr Complex Data	32 instr Decode	47 instr Video Graphics building blocks Advanced vector instr	8 instr String/XML processing POP-Count CRC	7 instr Encryption and Decryption Key Generation	~100 new instr. ~300 legacy sse instr updated 256-bit vector 3 and 4- operand instructions



Intel Advanced Vector eXtensions

AVX also supported by AMD processors

2011

2012

2013

2014

2015

Future

Comp

AVX Registers getting wider, instruction set getting richer

n & Weaver

87 GFLOPS

185 GFLOPS

~225 GFLOPS

~500 GFLOPS

tbd GFLOPS

tbd GFLOPS

Westmere

Sandy
Bridge

Ivy Bridge

Haswell

Broadwell

Skylake

32 nm
SSE 4.2
DDR3
PCIe2

32 nm
AVX
(256 bit
registers)

DDR3
PCIe3

22 nm

22 nm
AVX2
(new
instructions)

DDR4
PCIe3

14 nm

14 nm
AVX 3.2
(512 bit
registers)

DDR4
PCIe4

<https://chrisadkin.io/2015/06/04/under-the-hood-of-the-batch-engine-simd-with-sql-server-2016-ctp/>



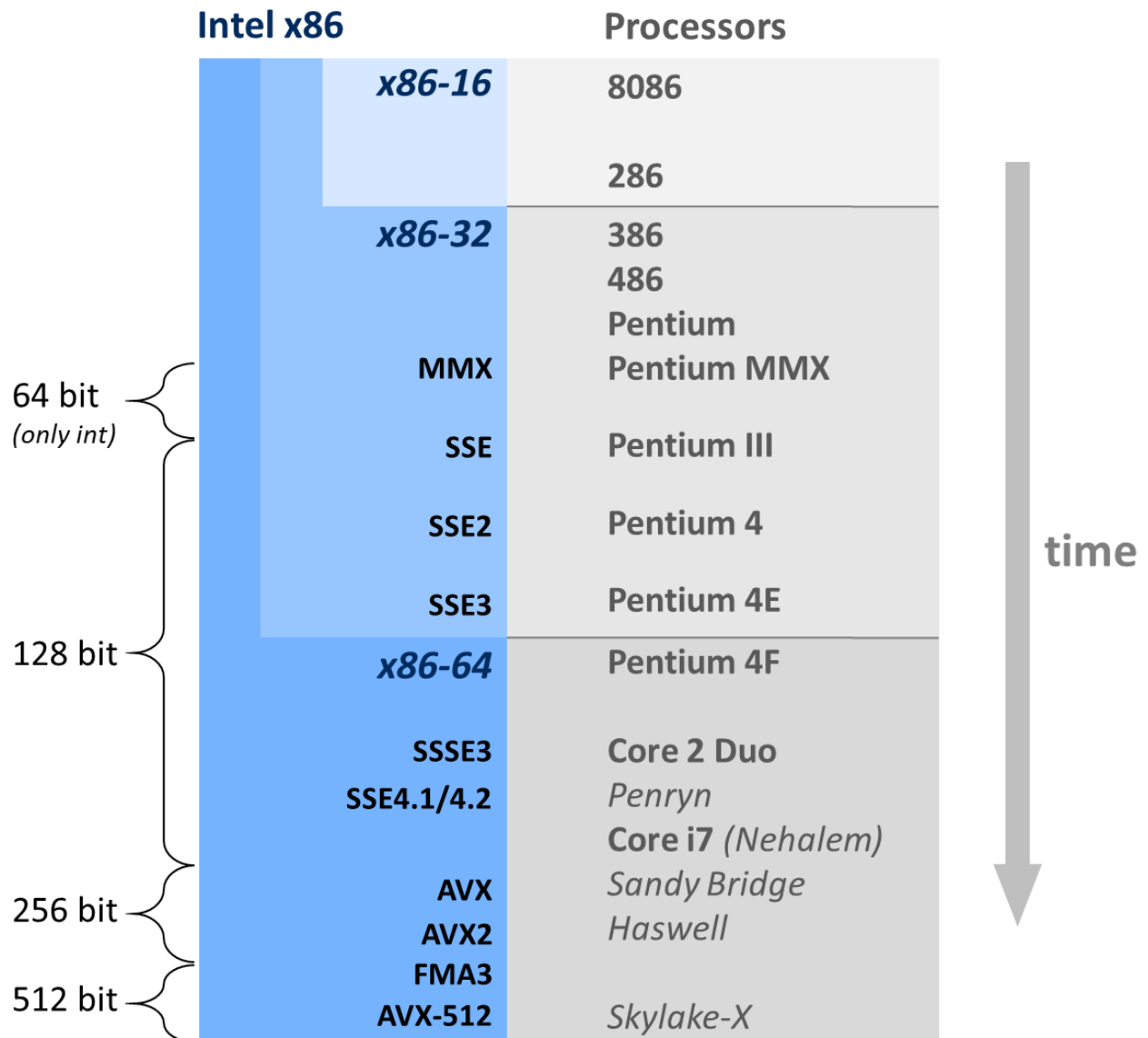
Evolution of Intel Vector Instructions

MMX:
Multimedia extension

SSE:
Streaming SIMD extension

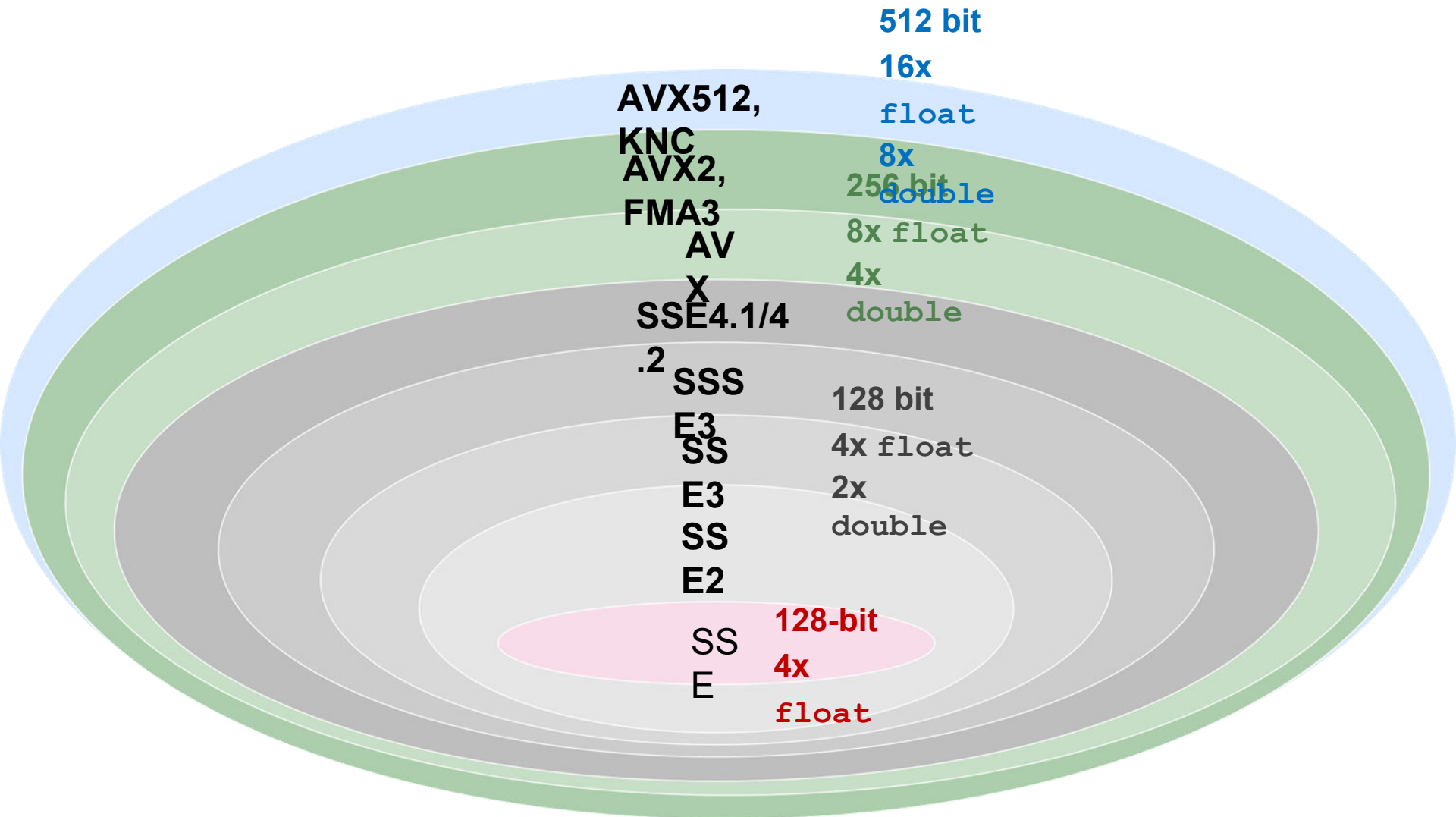
AVX:
Advanced vector extensions

*register
width*



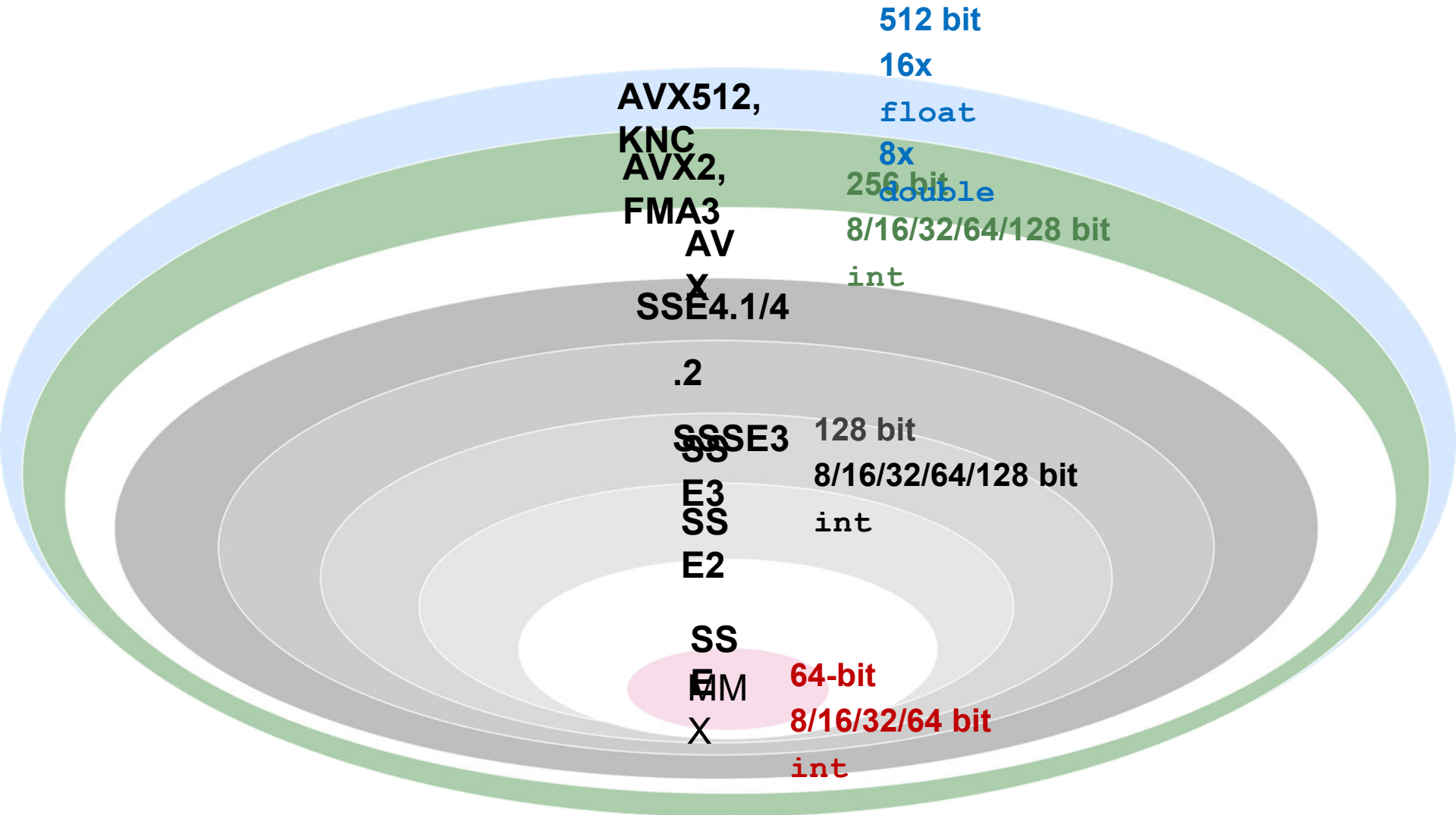


Intel SSE/AVX: Floating Point





Intel MMX/SSE/AVX: Integer





Other Types of Special Instructions

- Cryptography
- Bit-manipulation
- Cache/TLB control
- Debug support
- bounds checks
- Hardware random number generator
- String operations
- Half precision (FP16) support
- Machine control register access

```
unsigned int _mm_crc32_u16 (unsigned int crc, unsigned short v)
```

crc32

Synopsis

```
unsigned int _mm_crc32_u16 (unsigned int crc, unsigned short v)
#include <nmmintrin.h>
Instruction: crc32 r32, r16
CPUID Flags: SSE4.2
```

Description

Starting with the initial value in `rcr`, accumulates a CRC32 value for unsigned 16-bit integer `v`, and stores the result in `dst`.

Operation

```
tmp1[15:0] := v[0:15] // bit reflection
tmp2[31:0] := crc[0:31] // bit reflection
tmp3[47:0] := tmp1[15:0] << 32
tmp4[47:0] := tmp2[31:0] << 16
tmp5[47:0] := tmp3[47:0] XOR tmp4[47:0]
tmp6[31:0] := tmp5[47:0] MOD2 0x11EDC6F41
dst[31:0] := tmp6[0:31] // bit reflection
```



SSE3/4/AVX128 XMM Register File

*128 bit = 2 doubles = 4
singles*

%xmm0	%xmm8
%xmm1	%xmm9
%xmm2	%xmm10
%xmm3	%xmm11
%xmm4	%xmm12
%xmm5	%xmm13
%xmm6	%xmm14
%xmm7	%xmm15

Since SSE2 also vectors of 8/16/32/64/128-bit integers



AVX Register File (Sandy Bridge and Later)

256 bit = 4 doubles = 8
singles

%ymm0	%ymm8
%ymm1	%ymm9
%ymm2	%ymm10
%ymm3	%ymm11
%ymm4	%ymm12
%ymm5	%ymm13
%ymm6	%ymm14
%ymm7	%ymm15

Since AVX2 also vectors of 8/16/32/64/128/256-bit integers



AVX512 Register File

512 bit = 16 doubles = 32
singles

%zmm0	%zmm8	%zmm16	%zmm24
%zmm1	%zmm9	%zmm17	%zmm25
%zmm2	%zmm10	%zmm18	%zmm26
%zmm3	%zmm11	%zmm19	%zmm27
%zmm4	%zmm12	%zmm20	%zmm28
%zmm5	%zmm13	%zmm21	%zmm29
%zmm6	%zmm14	%zmm22	%zmm30
%zmm7	%zmm15	%zmm23	%zmm31

Also vectors of 8/16/32/64/128/256/512-bit integers



XMM/YMM/ZMM

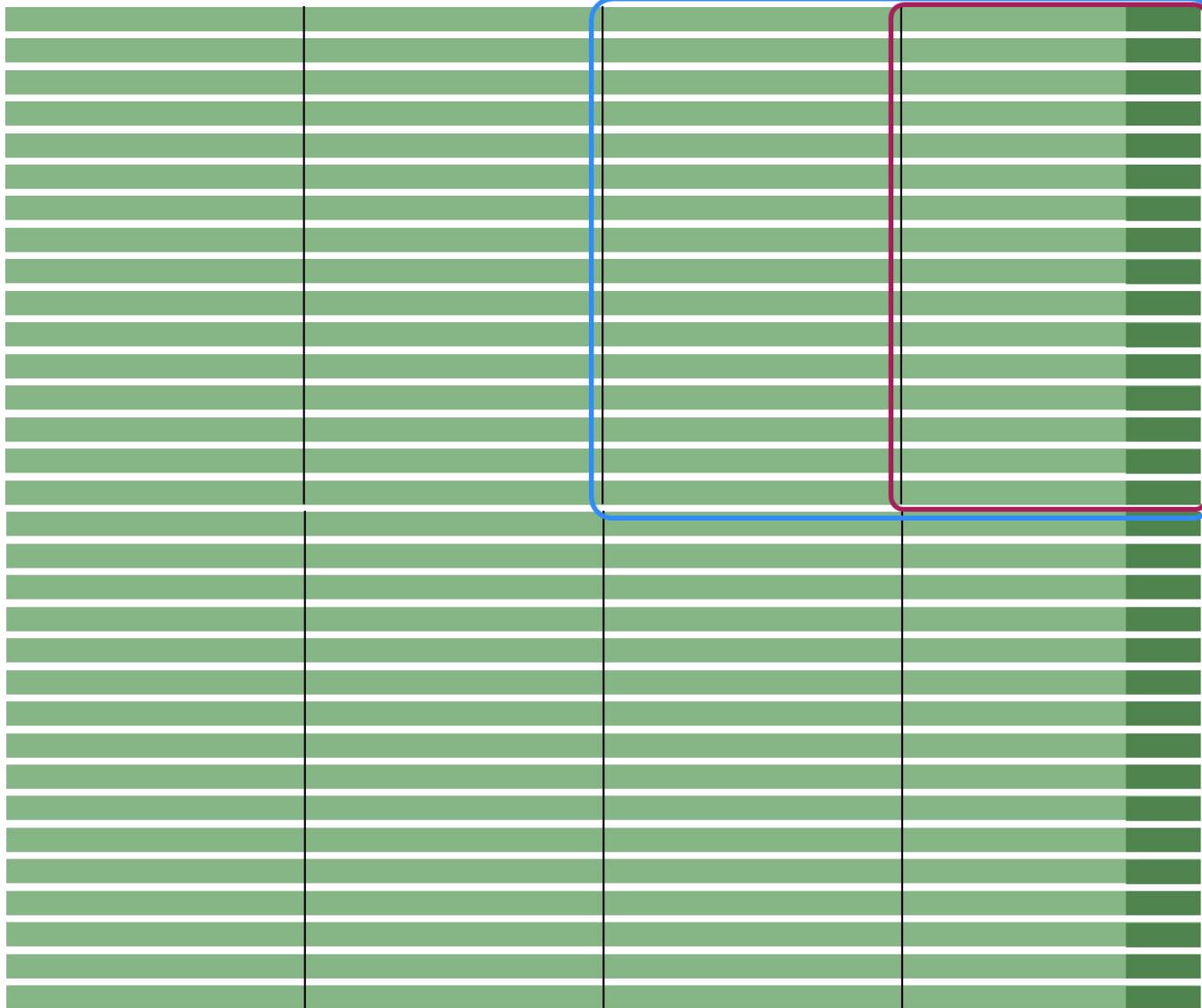
Relationship

32 zmm (AVX-512)

16 ymm (AVX)

16 xmm (SSE)

scalar



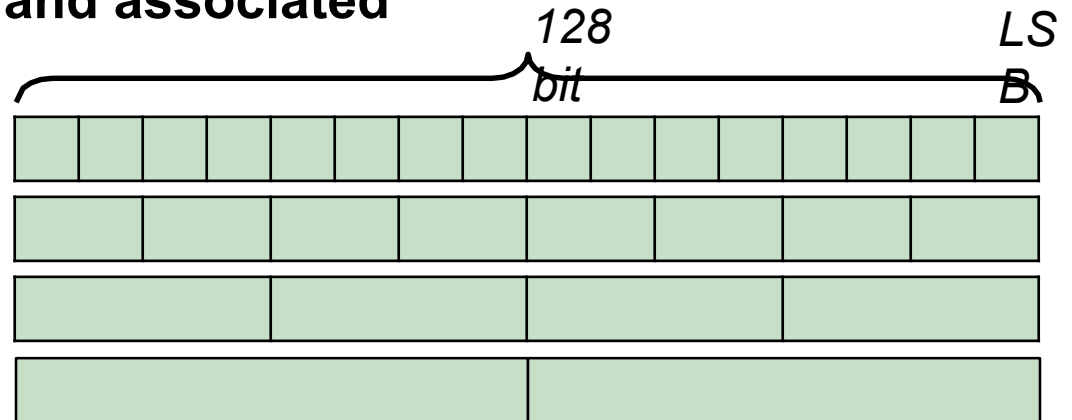


Detail: XMM/SSE2/3/4 Register

□ Different data types and associated instructions

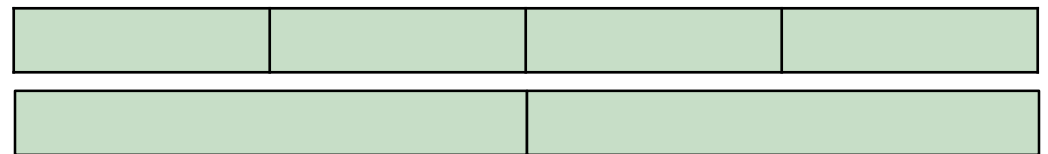
□ Integer vectors:

- 16-way byte
- 8-way 2 bytes
- 4-way 4 bytes
- 2-way 8 bytes



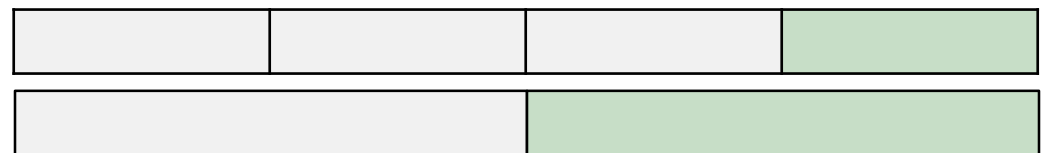
□ Floating point vectors:

- 4-way single (since SSE)
- 2-way double (since SSE2)



□ Floating point scalars:

- single (since SSE)
- double (since SSE2)





x86 Intrinsics AVX Data Types

Intrinsics: Direct access to assembly from C

Type	Meaning
<code>__m256</code>	256-bit as eight single-precision floating-point values, representing a YMM register or memory location
<code>__m256d</code>	256-bit as four double-precision floating-point values, representing a YMM register or memory location
<code>__m256i</code>	256-bit as integers, (bytes, words, etc.)
<code>__m128</code>	128-bit single precision floating-point (32 bits each)
<code>__m128d</code>	128-bit double precision floating-point (64 bits each)



Intrinsics AVX Code Nomenclature

Marking	Meaning
[s/d]	Single- or double-precision floating point
[i/u]nnn	Signed or unsigned integer of bit size <i>nnn</i> , where <i>nnn</i> is 128, 64, 32, 16, or 8
[ps/pd/sd]	Packed single, packed double, or scalar double
epi32	Extended packed 32-bit signed integer
si256	Scalar 256-bit integer



SIMD Vector Instructions in a Nutshell

□ What are these instructions?

- Extension of the ISA. Data types and instructions for parallel computation on short (2-16) vectors of integers and floats



+



x



4-
way

□ Why are they here?

- **Useful:** Many applications (e.g., multi media) feature the required fine grain parallelism – code potentially faster
- **Doable:** Chip designers have enough transistors available, easy to implement

SIMD = Single Instruction Multiple Data



SSE4/AVX2 Data Types

□ SSE4

data types

```
__m128 f;           // = {float f0, f1, f2, f3}
__m128d d;          // = {double d0, d1}
__m128i i;          // 16 8-bit, 8 16-bit, 4 32-bit, or 2 64-bit ints
```

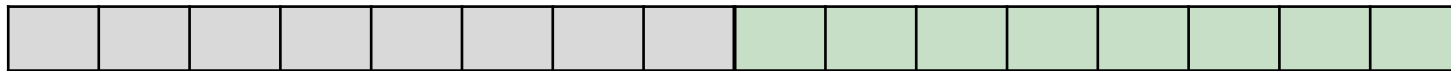
□ AVX2

data types

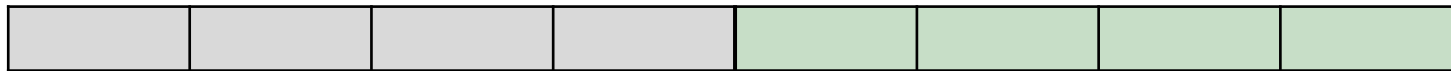
```
__m256 f;           // = {float f0, f1, ..., f7}
__m256d d;          // = {double d0, d1, d2, d3}
__m256i i;          // 32 8-bit, 16 16-bit, 8 32-bit, or 4 64-bit ints
__m128i i;
```



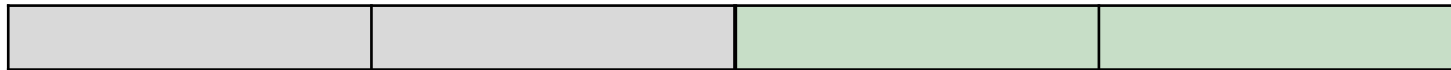
int8



int16



int32 or float



int64 or



SSE4/AVX128 4 x float Intrinsics

□ Instructions

- Naming convention: `_mm_<intrin_op>_<suffix>`
- Example:

```
// a is 16-byte aligned  
float a[4] = {1.0, 2.0, 3.0, 4.0};  
__m128 t = _mm_load_ps(a);
```

*p: packed
s: single
precision*

LSB

1.0	2.0	3.0	4.0
-----	-----	-----	-----

- Same result

as

```
_mm128 t = _mm_set_ps(4.0, 3.0, 2.0, 1.0)
```



AVX2 4 x double Intrinsics

□ Instructions

- Naming convention: `_mm256_<intrin_op>_<suffix>`
- Example:

```
// a is 32-byte aligned  
double a[4] = {1.0, 2.0, 3.0, 4.0};  
__m256d t = _mm256_load_pd(a);
```

p: packed
d: double
precision

LSB

1.0	2.0	3.0	4.0
-----	-----	-----	-----

- Same result

as

```
_mm256d t = _mm256_set_pd(4.0, 3.0, 2.0, 1.0)
```



Intrinsics

- **Assembly coded C functions**
- **Expanded inline upon compilation: no overhead**
- **Like writing assembly inside C**
- **Floating point:**
 - Intrinsics for basic operations (add, mult, ...)
 - Intrinsics for math functions: log, sin, ...
- **Our introduction is based on icc**
 - Most intrinsics work with gcc and Visual Studio (VS)
 - Some language extensions are icc (or even VS) specific

<https://software.intel.com/sites/landingpage/IntrinsicsGuide/>

Technologies

- ☐ MMX
- ☐ SSE
- ☐ SSE2
- ☐ SSE3
- ☐ SSSE3
- ☐ SSE4.1
- ☐ SSE4.2
- ☐ AVX
- ☐ AVX2
- ☐ FMA
- ☐ AVX-512
- ☐ KNC
- ☐ SVML
- ☐ Other

Categories

- ☐ Application-Targeted
- ☐ Arithmetic
- ☐ Bit Manipulation
- ☐ Cast
- ☐ Compare
- ☐ Convert
- ☐ Cryptography
- ☐ Elementary Math

Functions

- ☐ General Support
- ☐ Load
- ☐ Logical
- ☐ Mask
- ☐ Miscellaneous
- ☐ Move
- ☐ OS-Targeted
- ☐ Probability/Statistics
- ☐ Random
- ☐ Set
- ☐ Shift
- ☐ Special Math Functions

```

__m512i _mm512_4dpwssd_epi32 (_m512i src, _m512i a0, _m512i a1, _m512i a2, _m512i a3, _m128i * b) vp4dpwssd
__m512i _mm512_mask_4dpwssd_epi32 (_m512i src, _mmask16 k, _m512i a0, _m512i a1, _m512i a2,
_m512i a3, _m128i * b) vp4dpwssd
__m512i _mm512_maskz_4dpwssd_epi32 (_mmask16 k, _m512i src, _m512i a0, _m512i a1, _m512i a2,
_m512i a3, _m128i * b) vp4dpwssd
__m512i _mm512_4dpwssds_epi32 (_m512i src, _m512i a0, _m512i a1, _m512i a2, _m512i a3, _m128i * b) vp4dpwssds
__m512i _mm512_mask_4dpwssds_epi32 (_m512i src, _mmask16 k, _m512i a0, _m512i a1, _m512i a2,
_m512i a3, _m128i * b) vp4dpwssds
__m512i _mm512_maskz_4dpwssds_epi32 (_m512i src, _mmask16 k, _m512i a0, _m512i a1, _m512i a2,
_m512i a3, _m128i * b) vp4dpwssds
__m512 _mm512_4fmadd_ps (_m512 a, _m512i b0, _m512i b1, _m512i b2, _m512i b3, _m128i * c) v4fmaddps
__m512 _mm512_mask_4fmadd_ps (_m512 a, _mmask16 k, _m512i b0, _m512i b1, _m512i b2, _m512i b3,
_m128i * c) v4fmaddps
__m512 _mm512_maskz_4fmadd_ps (_m512 a, _mmask16 k, _m512i b0, _m512i b1, _m512i b2, _m512i b3,
_m128i * c) v4fmaddps
__m128 _mm_4fmadd_ss (__m128 a, __m128 b0, __m128 b1, __m128 b2, __m128 b3, __m128 * c) v4fmaddss
__m128 _mm_mask_4fmadd_ss (__m128 a, __mmask8 k, __m128 b0, __m128 b1, __m128 b2, __m128 b3,
__m128 * c) v4fmaddss
__m128 _mm_maskz_4fmadd_ss (__m128 a, __mmask8 k, __m128 b0, __m128 b1, __m128 b2, __m128 b3,
__m128 * c) v4fmaddss
__m512 _mm512_4fnmadd_ps (_m512 a, _m512i b0, _m512i b1, _m512i b2, _m512i b3, _m128i * c) v4fnmaddps
__m512 _mm512_mask_4fnmadd_ps (_m512 a, _mmask16 k, _m512i b0, _m512i b1, _m512i b2, _m512i b3,
_m128i * c) v4fnmaddps
__m512 _mm512_maskz_4fnmadd_ps (_m512 a, _mmask16 k, _m512i b0, _m512i b1, _m512i b2, _m512i b3,
_m128i * c) v4fnmaddps
__m128 _mm_4fnmadd_ss (__m128 a, __m128 b0, __m128 b1, __m128 b2, __m128 b3, __m128 * c) v4fnmaddss
__m128 _mm_mask_4fnmadd_ss (__m128 a, __mmask8 k, __m128 b0, __m128 b1, __m128 b2, __m128 b3,
__m128 * c) v4fnmaddss
__m128 _mm_maskz_4fnmadd_ss (__m128 a, __mmask8 k, __m128 b0, __m128 b1, __m128 b2, __m128 b3,
__m128 * c) v4fnmaddss
__m128i _mm_abs_epi16 (__m128i a) pabsw
__m128i _mm_mask_abs_epi16 (__m128i src, __mmask8 k, __m128i a) vpabsw
__m128i _mm_maskz_abs_epi16 (__mmask8 k, __m128i a) vpabsw

```



Note on AVX-512...

- 512 bit vectors may be more efficient
 - Twice as much operations per instruction
 - But also requires 2x the resources over
- Really heavy vector code is now done on the GPU
 - Thousands of simultaneous operations
- Intel is now disabling it on some of the newer designs
 - The big/little "Alder Lake":
 - The little cores don't support AVX-512



Laptop CPU Specs

```
$ sysctl -a | grep cpu
```

```
hw.physicalcpu: 4
```

```
hw.logicalcpu: 8
```

```
machdep.cpu.brand_string: Intel(R) Core(TM) i5-1038NG7 CPU @ 2.00GHz
```

```
machdep.cpu.features: FPU VME DE PSE TSC4 MSR PAE MCE CX8 APIC SEP MTRR PGE MCA CMOV  
PAT PSE36 CLFSH DS ACPI MMX FXSR SSE SSE2 SS HTT TM PBE SSE3 PCLMULQDQ DTES64 MON  
DSCPL VMX EST TM2 SSSE3 FMA CX16 TPR PDCM SSE4.1 SSE4.2 x2APIC MOVBE POPCNT AES PCID  
XSAVE OSXSAVE SEGLIM64 TSCTMR AVX1.0 RDRAND F16C
```

```
machdep.cpu.leaf7 features: RDWRFSGS TSC THREAD OFFSET SGX BMI1 AVX2 FDPEO SMEP BMI2  
ERMS INVPCID FPU CS DS AVX512F AVX512DQ RDSEED ADX SMAP AVX512IFMA CLFSOPT IPT  
AVX512CD SHA AVX512BW AVX512VL AVX512VBMI UMIP PKU GFNI VAES VPCLMULQDQ AVX512VNNI  
AVX512BITALG AVX512VPOPCNTDQ RDPID SGXLC FSREPMOV MDCLEAR IBRS STIBP L1DF ACAPMSR  
SSBD
```

```
machdep.cpu.extfeatures: SYSCALL XD 1GBPAGE EM64T LAHF LZCNT PREFETCHW RDTSCP TSCI
```



Example

```
#include <smmintrin.h>
```

```
#include <immintrin.h>
```

<https://www.cs.virginia.edu/~cr4bd/3330/F2018/simdref.html>

```
int add_no_AVX(int size, int *first_array, int *second_array) {
    for (int i = 0; i < size; ++i) {
        first_array[i] += second_array[i];
    }
}

int add_AVX(int size, int *first_array, int *second_array) {
    int i = 0;
    for (; i + 8 <= size; i += 8) {
        // load 256-bit chunks of each array
        __m256i first_values = _mm_loadu_si256((__m256i*) &first_array[i]);
        __m256i second_values = _mm_loadu_si256((__m256i*) &second_array[i]);

        // add each pair of 32-bit integers in the 256-bit chunks
        first_values = _mm256_add_epi32(first_values, second_values);

        // store 256-bit chunk to first array
        _mm_storeu_si256((__m256i*) &first_array[i], first_values);
    }
    // handle left-over
    for (; i < size; ++i) {
        first_array[i] += second_array[i];
    }
}
```



Example converting to SSE

<https://www.cs.virginia.edu/~cr4bd/3330/F2018/simdref.html>

```
#include <smmintrin.h>
#include <immintrin.h>
```

```
int add_SSE(long size, int *first_array, int *second_array) {
    int i = 0;
    for (; i + 8 <= size; i += 8) {
        // load 256-bit chunks of each array
        __m128i first_values = _mm_loadu_si128((__m128i*) &first_array[i]);
        __m128i second_values = _mm_loadu_si128((__m128i*) &second_array[i]);

        // add each pair of 32-bit integers in the 256-bit chunks
        first_values = _mm_add_epi32(first_values, second_values);

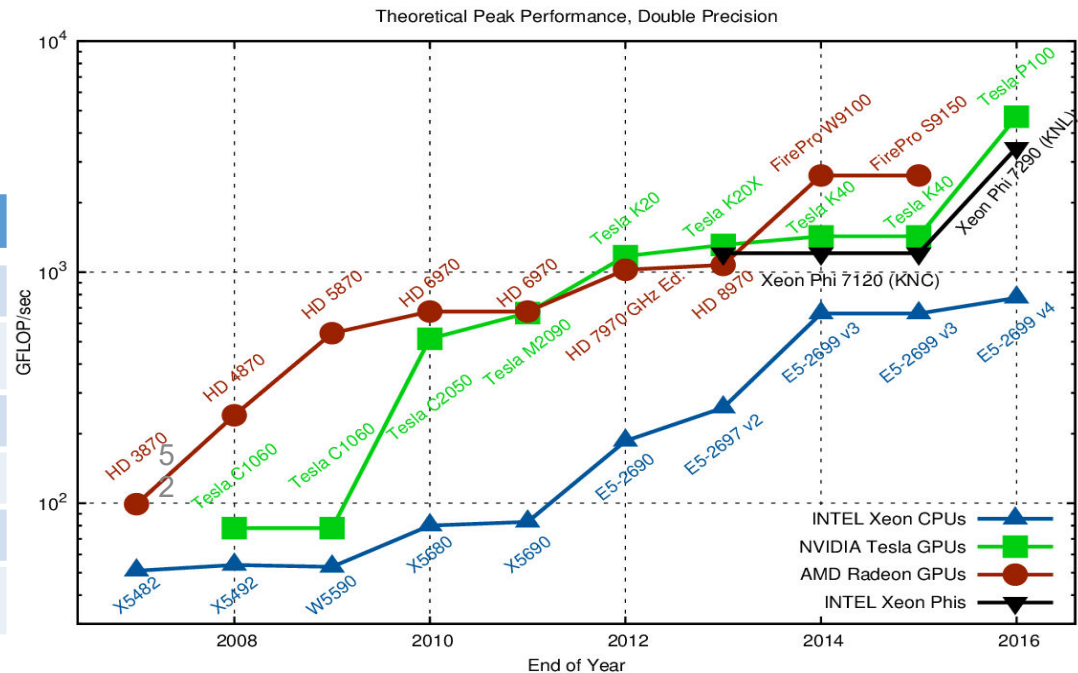
        // store 256-bit chunk to first array
        _mm_storeu_si128((__m128i*) &first_array[i], first_values);
    }
    // handle left-over
    for (; i < size; ++i) {
        first_array[i] += second_array[i];
    }
}
```





Raw Double-Precision Throughput

Characteristic	Value
CPU	i7-5557U
Clock rate (sustained)	3.1 GHz
Instructions per clock (mul_pd)	2
Parallel multiplies per instruction	4
Peak double FLOPS	24.8 GFLOPS



<https://www.karlrupp.net/2013/06/cpu-gpu-and-mic-hardware-characteristics-over-time/>

Actual performance is lower because of overhead



Vectorized Matrix Multiplication

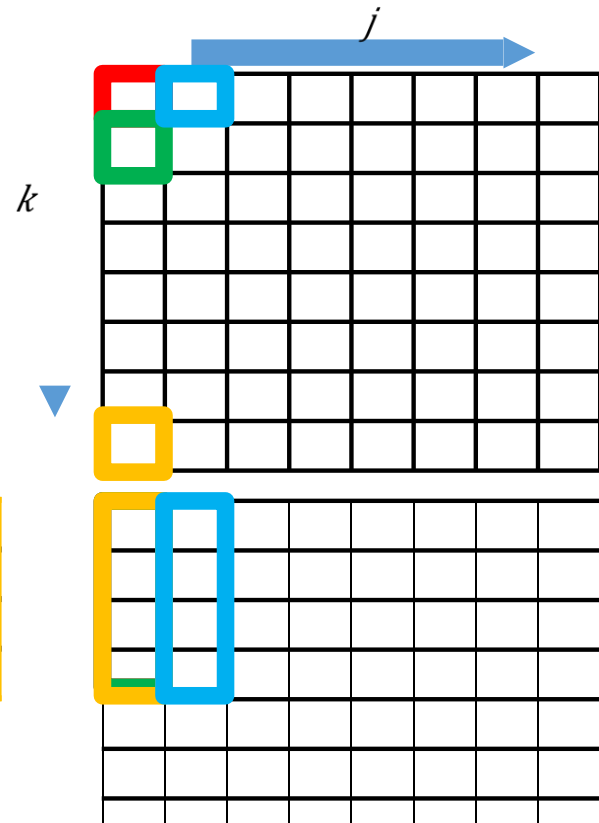
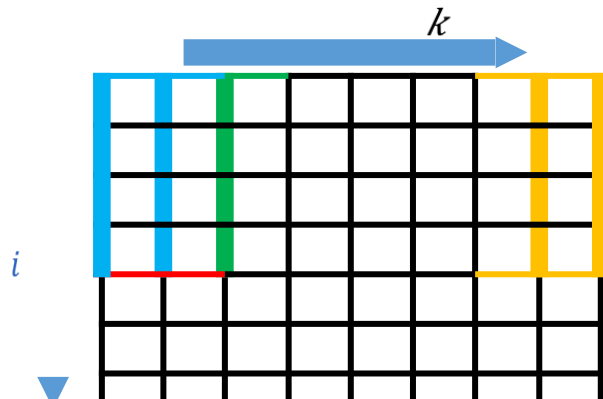
for i ...; **i+=4**

i += 4

Inner Loop:

for j

```
__m256d c0 = {0,0,0,0};  
for (int k=0; k<N; k++) {  
    c0 = _mm256_fmadd_pd(  
        _mm256_load_pd(a+i+k*N),  
        _mm256_broadcast_sd(b+k*j*N),  
        c0);  
}  
_mm256_store_pd(c+i+j*N, c0);
```





“Vectorized” dgemm

```
void dgemm_avx(int N, double *a, double *b, double *c){
    int i,j,k; __m256d c0;
    for(i = 0; i < N; i += 4){
        for(j = 0; j < N; ++j){
            c0 = {0,0,0,0}
            for(k = 0; k < N; ++k){
                c0 = mm256_add_pd(
                    c0,
                    mm256_mul_pd(
                        mm256_load_pd(a+i+k*N),
                        mm256_broadcast_sd(b+k+j*N)) );
            }
            _mm256_store_pd(c+i+j*N, c0);
        }
    }
}
```



Performance

N	Gflops	
	scalar	avx
32	1.30	4.56
160	1.30	5.47
480	1.32	5.27
960	0.91	3.64

- 4x faster
- But still << theoretical 25 GFLOPS!



Other SIMD/Vector Hardware

- **Original SIMD machines (CM-2,...)**
 - Don't really have anything in common with SIMD vector extension
- **Vector Computers (NEC SX6, Earth simulator, SX-Aurora)**
 - Vector lengths of up to 128
 - High bandwidth memory, no memory hierarchy
 - Pipelined vector operations
- **Very long instruction word (VLIW) architectures**
 - Explicit parallelism
 - More flexible
 - No data reorganization necessary
- **SIMT (Nvidia)**
 - SIMD lane == tread
 - Predicated execution, scheduling,...



Any Questions?

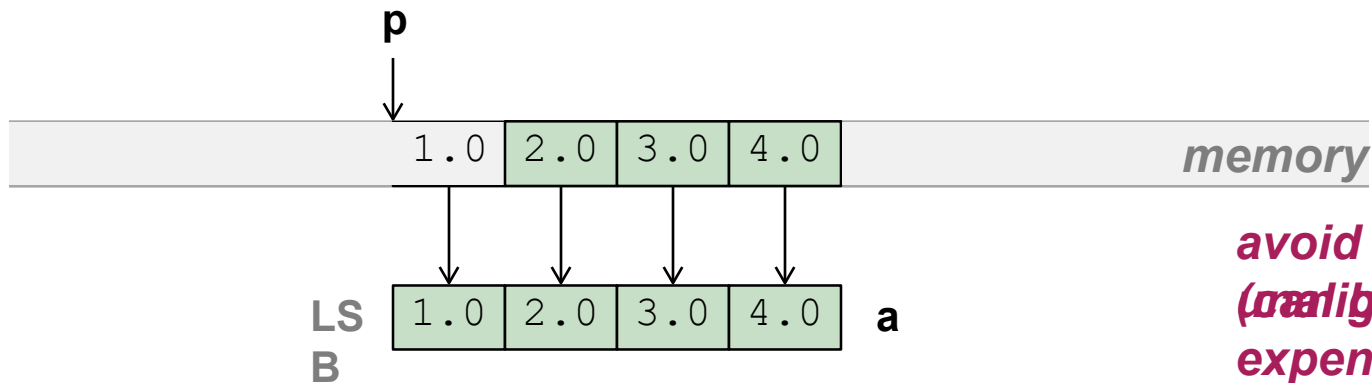


SSE4/AVX128 and AVX2 Intrinsics

- **Load and store**
- **Constants**
- **Arithmetic**
- **Comparison**
- **Conversion**
- **Shuffles**



Loads and Stores



*avoid
(~~unaligned~~
expensive) on
possibly
penalty*

```
a = _mm_load_ps(p);    // p 16-byte aligned
```

```
a = _mm_loadu_ps(p);  // p not aligned
```

```
a = _mm256_load_pd(p);    // p 32-byte aligned
```

```
a = _mm256_loadu_pd(p);  // p not aligned
```

load_ps/pd on unaligned pointer: seg
fault



Example: Load and Set in SSE4/AVX128 and AVX2

Intrinsic Name	Operation	Corresponding SSE/AVX Instructions
_mm256_broadcast_pd	Broadcast 128 bits from memory to all elements of dst	vbroadcastf128
_mm_i32gather_epi32	Gather 32-bit integers from memory using 32-bit indices	vpgatherdd
_mm_load_ss	Load the low value and clear the three high values	movss
_mm256_loadu2_m128i	Load two 128-bit values from memory, and combine them into dst	composite
_mm256_load_ps	Load eight values, address aligned	vmovaps
_mm_loadu_ps	Load four values, address unaligned	movups
_mm_maskload_pd	Load packed double-precision elements from memory using mask	vmaskmovpd
Intrinsic Name	Operation	Corresponding SSE/AVX Instruction
_mm_set_ss	Set the low value and clear the three high values	Composite
_mm_set1_ps	Set all four words with the same value	Composite
_mm_set_ps	Set four values	Composite
_mm_setr_ps	Set four values, in reverse order	Composite
_mm_setzero_ps	Clear all four values	Composite
_mm256_set1_pd	Set all four words with the same value	Composite
_mm256_set_ps	Set eight values	Composite
_mm256_set1_epi64x	Broadcast 64-bit integer a to all elements of dst	Composite
_mm256i_setzero_si256	Clear all 256 bits	Composite



Stores Less Powerful than Loads

Intrinsic Name	Operation	Corresponding SSE/AVX Instructions
_mm256_store_pd	Store 4 doubles to aligned memory	vmovapd
_mm_store_pd	Store 2 doubles to aligned memory	movapd
_mm256_maskstore_ps	Store single-precision elements from a into memory using mask	vmaskmovps
_mm256_stream_si256	Non-temporal store	vmovntdq
_mm256_storeu2_m128d	Store the high and low 128-bit into memory two different locations	composite
_mm_storel_epi64	Store 64 bit of XMM register to memory	movq
_mm_store1_pd	Store lowest double to memory	composite

Constants

LS
B

1.0	2.0	3.0	4.0
-----	-----	-----	-----

a

```
a = _mm_set_ps(4.0, 3.0, 2.0, 1.0);
```

LS
B

1.0	1.0	1.0	1.0
-----	-----	-----	-----

b

```
b = _mm_set1_ps(1.0);
```

LS
B

1.0	0	0	0
-----	---	---	---

c

```
c = _mm_set_ss(1.0);
```

LS
B

0	0	0	0
---	---	---	---

d

```
d = _mm_setzero_ps();
```



Arithmeti

C

SSE/SS

E2

Intrinsic Name	Operation	Corresponding SSE Instruction
_mm_add_ss	Addition	ADDSS
_mm_add_ps	Addition	ADDPS
_mm_sub_ss	Subtraction	SUBSS
_mm_sub_ps	Subtraction	SUBPS
_mm_mul_ss	Multiplication	MULSS
_mm_mul_ps	Multiplication	MULPS
_mm_div_ss	Division	DIVSS
_mm_div_ps	Division	DIVPS
_mm_sqrt_ss	Squared Root	SQRTSS
_mm_sqrt_ps	Squared Root	SQRTPS
_mm_rcp_ss	Reciprocal	RCPSS
_mm_rcp_ps	Reciprocal	RCPPS
_mm_rsqrt_ss	Reciprocal Squared Root	RSQRTSS
_mm_rsqrt_ps	Reciprocal Squared Root	RSQRTPS
_mm_min_ss	Computes Minimum	MINSS
_mm_min_ps	Computes Minimum	MINPS
_mm_max_ss	Computes Maximum	MAXSS
_mm_max_ps	Computes Maximum	MAXPS

SS

E3

Intrinsic Name	Operation	Corresponding SSE3 Instruction
_mm_addsub_ps	Subtract and add	ADDSUBPS
_mm_hadd_ps	Add	HADDPS
_mm_hsub_ps	Subtracts	HSUBPS

AV

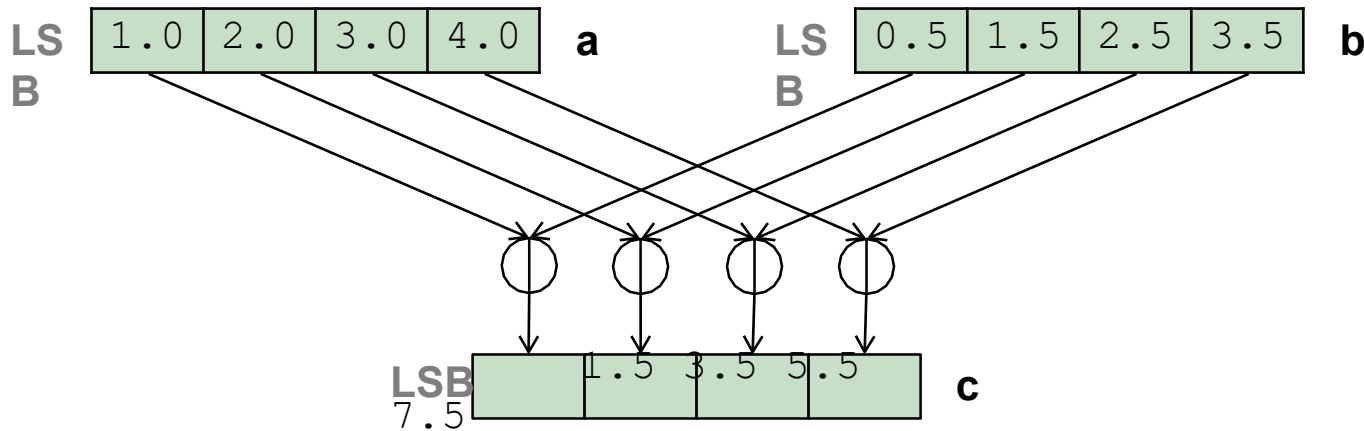
Xtrinsic

Xtrinsic	Operation	Corresponding SSE4 Instruction
_mm256_add_pd	Addition	vaddpd
_mm256_addsub_pd	Subtract and add	vaddsubpd
_mm256_hadd_pd	Horizontal add	vhaddpd
_mm256_fmaddsub_pd	Fused multiply add	vfmaddsub132pd vfmaddsub213pd vfmaddsub231pd
_mm256_fmsub_pd	Fused multiply subtract	vfmsub132pd vfmsub213pd vfmsub231pd



Arithmeti

c



```
c = _mm_add_ps(a, b);
```

```
c = _mm256_add_pd(a, b);
```

analogou

analogou

```
c = _mm_sub_ps(a, b);
```

```
c = _mm256_sub_pd(a, b);
```

```
c = _mm_mul_ps(a, b);
```

```
c = _mm256_mul_pd(a, b);
```



Example:

AVX

```
void addindex(double *x, int n) { for (int i
    = 0; i < n; i++)
    x[i] = x[i] + i;
}
```

```
#include <immintrin.h>
```

```
// n a multiple of 4, x is 32-byte aligned void
```

```
addindex_vec(double *x, int n) {
    __m256d index, x_vec;
```

```
    for (int i = 0; i < n; i+=4) {
```

```
        x_vec = _mm256_load_pd(x+i);           // load 4 double
```

```
        index = _mm256_set_pd(i+3, i+2, i+1, i); // create vector with indexes x_vec
```

```
        = _mm256_add_pd(x_vec, index);         // add the two
```

```
        _mm256_store_pd(x+i, x_vec);          // store back
```

```
    }
```

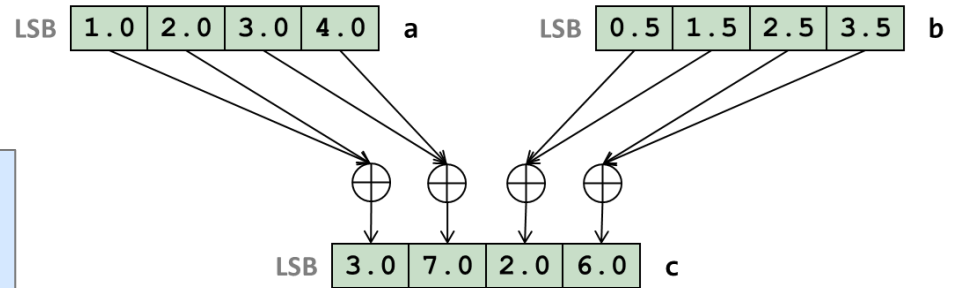
```
}
```



Example: SSE4

// n is even

```
void lp(float *x, float *y, int n) { for (int i = 0; i
    < n/2; i++)
    y[i] = (x[2*i] + x[2*i+1])/2;
}
```



```
c = _mm_hadd_ps(a, b);
```

```
c = _mm_hsub_ps(a, b);
```

```
#include <ia32intrin.h>
```

// n a multiple of 8, x, y are 16-byte aligned void

```
lp_vec(float *x, int n) {
    __m128 half, v1, v2, avg;
```

```
half = _mm_set1_ps(0.5); // set vector to all 0.5 for(int i = 0; i <
n/8; i++) {
```

```
    v1 =
```

```
    _mm_load_ps(x+i*8); v2
```

```
    avg = _mm_hadd_ps(v1, v2); avg
```

```
    = _mm_load_ps(x+i*8);
```

```
    _mm_store_ps(y+i*4, avg);
```

```
}
```

```
}
```

// load first 4 floats

// load next 4 floats

// add pairs of floats

// multiply with 0.5

// save result



Comparisons: SSE4/AVX128

Intrinsic Name	Operation	Corresponding SSE Instruction
_mm_cmpeq_ss	Equal	CMPEQSS
_mm_cmpeq_ps	Equal	CMPEQPS
_mm_cmplt_ss	Less Than	CMPLTSS
_mm_cmplt_ps	Less Than	CMPLTPS
_mm_cmple_ss	Less Than or Equal	CMPLESS
_mm_cmple_ps	Less Than or Equal	CMPLEPS
_mm_cmpgt_ss	Greater Than	CMPLTSS
_mm_cmpgt_ps	Greater Than	CMPLTPS
_mm_cmpge_ss	Greater Than or Equal	CMPLESS
_mm_cmpge_ps	Greater Than or Equal	CMPLEPS
_mm_cmpneq_ss	Not Equal	CMPNEQSS
_mm_cmpneq_ps	Not Equal	CMPNEQPS
_mm_cmpnlt_ss	Not Less Than	CMPNLTSS
_mm_cmpnlt_ps	Not Less Than	CMPNLTPS
_mm_cmpnle_ss	Not Less Than or Equal	CMPNLESS
_mm_cmpnle_ps	Not Less Than or Equal	CMPNLEPS
_mm_cmpngt_ss	Not Greater Than	CMPNLTSS
_mm_cmpngt_ps	Not Greater Than	CMPNLTPS
_mm_cmpnge_ss	Not Greater Than or Equal	CMPNLESS
_mm_cmpnge_ps	Not Greater Than or Equal	CMPNLEPS

Intrinsic Name	Operation	Corresponding SSE Instruction
_mm_cmpord_ss	Ordered	CMPORDSS
_mm_cmpord_ps	Ordered	CMPORDPS
_mm_cmpunord_ss	Unordered	CMPUNORDSS
_mm_cmpunord_ps	Unordered	CMPUNORDPS
_mm_comieq_ss	Equal	COMISS
_mm_comilt_ss	Less Than	COMISS
_mm_comile_ss	Less Than or Equal	COMISS
_mm_comigt_ss	Greater Than	COMISS
_mm_comige_ss	Greater Than or Equal	COMISS
_mm_comineq_ss	Not Equal	COMISS
_mm_ucomieq_ss	Equal	UCOMISS
_mm_ucomilt_ss	Less Than	UCOMISS
_mm_ucomile_ss	Less Than or Equal	UCOMISS
_mm_ucomigt_ss	Greater Than	UCOMISS
_mm_ucomige_ss	Greater Than or Equal	UCOMISS
_mm_ucomineq_ss	Not Equal	UCOMISS



Comparisons:

AVX

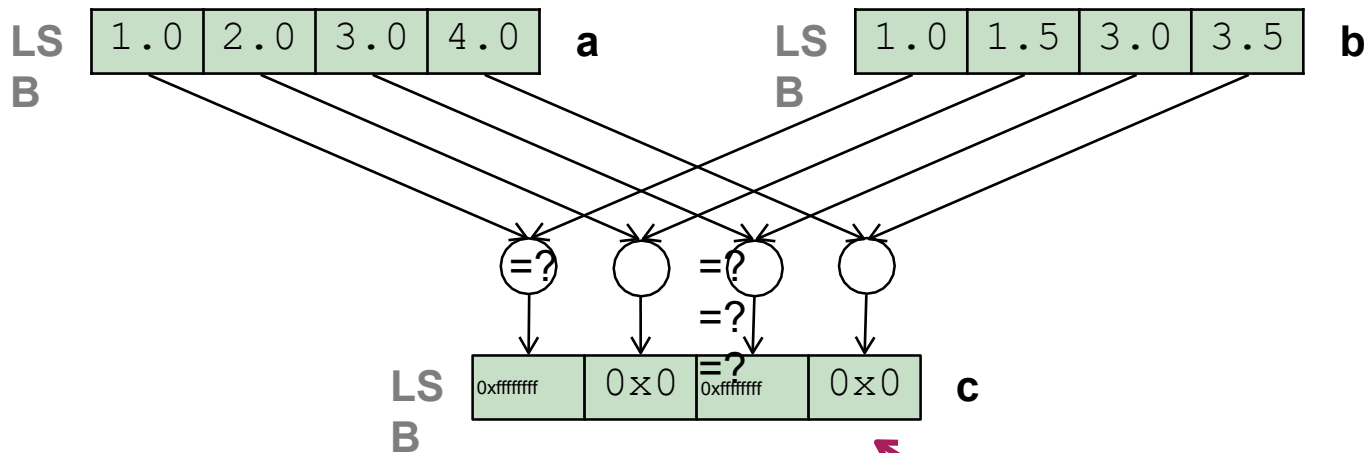
Intrinsic Name	Operation	Corresponding SSE Instruction
<code>_mm_cmp_pd</code>	Controlled by <code>imm8</code>	<code>vcmppd</code>
<code>_mm256_cmp_pd</code>	Controlled by <code>imm8</code>	<code>vcmppd</code>

imm8:

0: OP := <code>_CMP_EQ_OQ</code>	16: OP := <code>_CMP_EQ_OS</code>
1: OP := <code>_CMP_LT_OS</code>	17: OP := <code>_CMP_LT_OQ</code>
2: OP := <code>_CMP_LE_OS</code>	18: OP := <code>_CMP_LE_OQ</code>
3: OP := <code>_CMP_UNORD_Q</code>	19: OP := <code>_CMP_UNORD_S</code>
4: OP := <code>_CMP_NEQ_UQ</code>	20: OP := <code>_CMP_NEQ_US</code>
5: OP := <code>_CMP_NLT_US</code>	21: OP := <code>_CMP_NLT_UQ</code>
6: OP := <code>_CMP_NLE_US</code>	22: OP := <code>_CMP_NLE_UQ</code>
7: OP := <code>_CMP_ORD_Q</code>	23: OP := <code>_CMP_ORD_S</code>
8: OP := <code>_CMP_EQ_UQ</code>	24: OP := <code>_CMP_EQ_US</code>
9: OP := <code>_CMP_NGE_US</code>	25: OP := <code>_CMP_NGE_UQ</code>
10: OP := <code>_CMP_NGT_US</code>	26: OP := <code>_CMP_NGT_UQ</code>
11: OP := <code>_CMP_FALSE_OQ</code>	27: OP := <code>_CMP_FALSE_OS</code>
12: OP := <code>_CMP_NEQ_OQ</code>	28: OP := <code>_CMP_NEQ_OS</code>
13: OP := <code>_CMP_GE_OS</code>	29: OP := <code>_CMP_GE_OQ</code>
14: OP := <code>_CMP_GT_OS</code>	30: OP := <code>_CMP_GT_OQ</code>
15: OP := <code>_CMP_TRUE_UQ</code>	31: OP := <code>_CMP_TRUE_US</code>



Comparison Example: SSE4



```
c = _mm_cmpeq_ps(a, b);
```

analogou

```
c = _mm_cmple_ps(a, b);
```

```
c = _mm_cmplt_ps(a, b);
```

```
c = _mm_cmpge_ps(a, b);
```

etc

Each field:

0xffffffff if true

0x0 if false

Return type:

_____m128



SSE4 Code

Example

```
void fcond(float *x, size_t n) { int i;

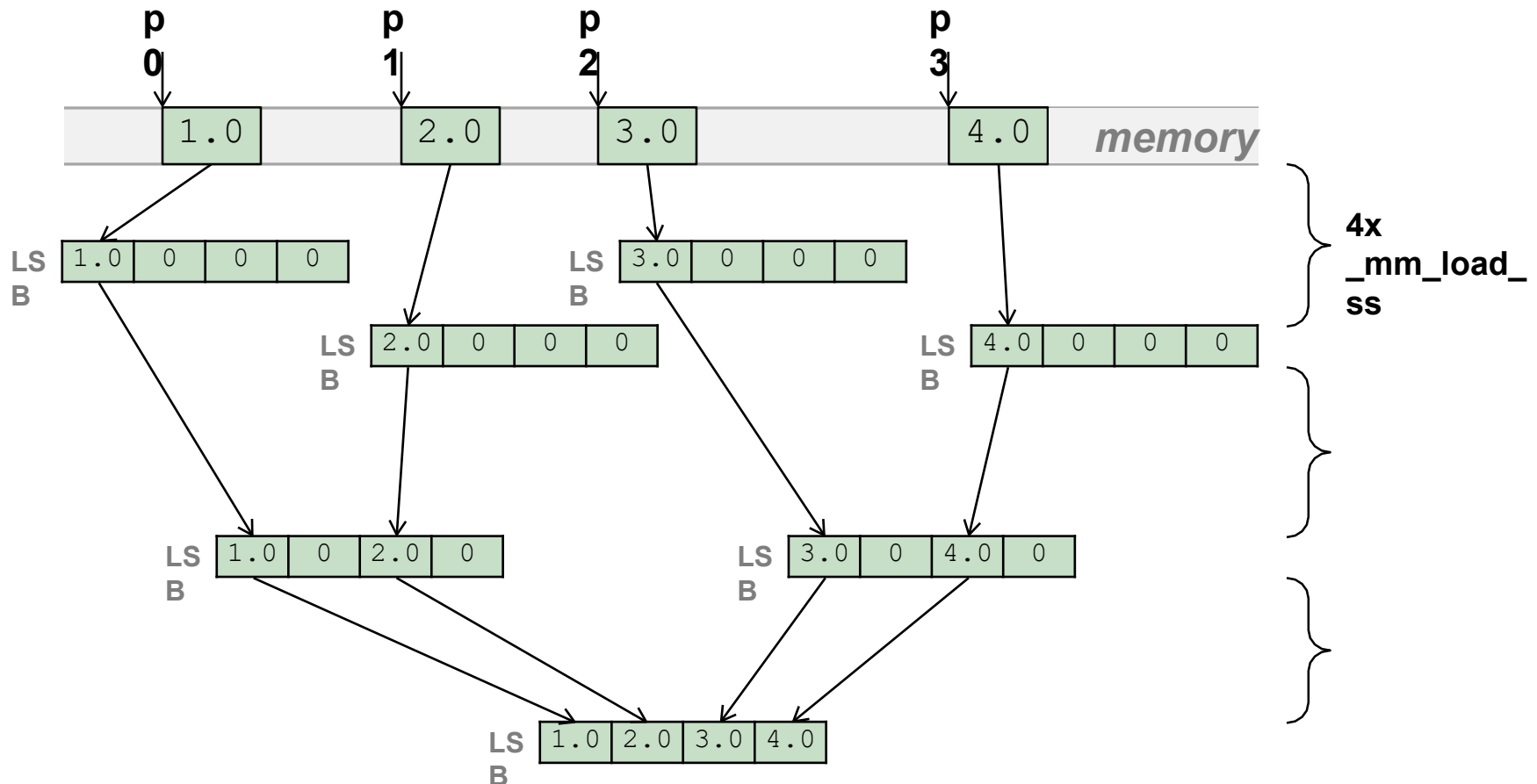
    for(i = 0; i < n; i++) { if(x[i] >
        0.5)
        x[i] += 1.;
        else x[i] -= 1.;
    }
}
```

```
#include <xmmintrin.h>
```

```
void fcond(float *a, size_t n) { int i;
    __m128 vt, vmask, vp, vm, vr, ones, mones, thresholds;
```

```
    ones      = _mm_set1_ps(1.);
    mone      = _mm_set1_ps(-1.);
    thresholds = _mm_set1_ps(0.5);
    for(i = 0; i < n; i+=4) {
        vt      = _mm_load_ps(a+i);
        vmask = _mm_cmpgt_ps(vt, thresholds); vp
            = _mm_and_ps(vmask, ones);
        vm      = _mm_andnot_ps(vmask, mones);
        vr      = _mm_add_ps(vt, _mm_or_ps(vp, vm));
        _mm_store_ps(a+i, vr);
    }
}
```

Example: Gather Operation



7 instructions, this is one good way of doing it



Code For Previous Slide

```
#include <ia32intrin.h>

__m128 LoadArbitrary(float *p0, float *p1, float *p2, float *p3) {
    __m128 a, b, c, d, e, f;

    a = _mm_load_ss(p0);
    b = _mm_load_ss(p1);
    c = _mm_load_ss(p2);
    d = _mm_load_ss(p3);
    e = _mm_shuffle_ps(a, b, _MM_SHUFFLE(1,0,2,0)); //only zeros are important
    f = _mm_shuffle_ps(c, d, _MM_SHUFFLE(1,0,2,0)); //only zeros are important
    return _mm_shuffle_ps(e, f, _MM_SHUFFLE(2,0,2,0));
}
```

With AVX2 and AVX-512 gather/scatter operations are introduced



How to Write Good Vector Code?

- **Take the “right” algorithm and the “right” data structures**
 - Fine grain parallelism
 - Correct alignment in memory
 - Contiguous arrays
- **Use a good compiler (e. g., vendor compiler)**
- **First: Try compiler vectorization**
 - Right options, pragmas and dynamic memory functions (Inform compiler about data alignment, loop independence,...)
 - Check generated assembly code *and* runtime
- **If necessary: Write vector code yourself**
 - Most expensive subroutine first
 - *Use intrinsics*, no (inline) assembly
 - *Important:* Understand the ISA



18-645: How to Write Fast Code

Tentative Course Calendar:

How To Write Fast Numerical Code: A Small Introduction

Srinivas Chellappa, Franz Franchetti, and Markus Püschel

Electrical and Computer Engineering
Carnegie Mellon University
{schellap, franzf, pueschel}@ece.cmu.edu

Abstract. The complexity of modern computing platforms has made it extremely difficult to write numerical code that achieves the best possible performance. Straightforward implementations based on algorithms that minimize the operations count often fall short in performance by at least one order of magnitude. This tutorial introduces the reader to a set of general techniques to improve the performance of numerical code, focusing on optimizations for the computer's memory hierarchy. Further, program generators are discussed as a way to reduce the implementation and optimization effort. Two running examples are used to demonstrate these techniques: matrix-matrix multiplication and the discrete Fourier transform.

1 Introduction

The growth in the performance of computing platforms in the past few decades has followed a reliable pattern usually referred to as Moore's Law. Moore observed in 1965 [1] that the number of transistors per chip roughly doubles every 18 months and predicted—correctly—that this trend would continue. In parallel, due to the shrinking size of transistors, CPU frequencies could be increased at roughly the same exponential rate. This trend has been the big supporter for many performance demanding applications in scientific computing (such as climate modeling and other physics simulations), consumer computing (such as audio, image, and video processing), and embedded computing (such as control, communication, and signal processing). In fact, these domains have a practically unlimited need for performance (for example, the ever growing need for higher resolution videos), and it seems that the evolution of computers is well on track to support these needs.

However, everything comes at a price, and in this case it is the increasing difficulty of writing the fastest possible software. In this tutorial, we focus on *numerical* software. By that we mean code that mainly consists of floating point computations.

The problem. To understand the problem we investigate Fig. 1, which considers various Intel architectures from the first Pentium to the (at the time of this writing) latest Core2 Extreme. The x -axis shows the year of release. The y -axis, in log-scale, shows both the CPU frequency (in MHz) and the single/double precision theoretical peak performance (in Mflop/s = Mega Floating point Operations per Second) of the respective machines.

Date	Day	Class Activity
August		
27	Mon.	Semester Begin – Overview of course
29	Wed.	Architecture and its impact on performance
September		
3	Mon.	Labor Day: No Classes
5	Wed.	Designing fast kernels
10	Mon.	Benchmarking
12	Wed.	Intro to SIMD Programming
17	Mon.	Compiler Optimizations
19	Wed.	Class Cancelled – Replaced with Project Kickoff Meetings
24	Mon.	The memory hierarchy
26	Wed.	Optimization for the memory hierarchy – Part 1
October		
1	Mon.	Optimization for the memory hierarchy – Part 2
3	Wed.	Intro to Explicit Parallelism
8	Mon.	Collective Communications – Part 1
10	Wed.	Class Cancelled – Replaced with Project Midterm Review Meetings
15	Mon.	Collective Communications – Part 2
17	Wed.	Designing Efficient Communication Patterns – Part 1
19	Fri.	Mid-Semester Break: No Classes
22	Mon.	Designing Efficient Communication Patterns – Part 2
24	Wed.	Shared Memory Parallelism
29	Mon.	From Shared Memory to Heterogeneous Architectures
31	Wed.	Introduction to Algorithm Design
November		
5	Mon.	Putting everything together
7	Wed.	Class Cancelled – Replaced with Project Final Review Meetings
12	Mon.	Midterm Review
14	Wed.	Midterm Exam
19	Mon.	Project Meeting Time
21-23	W-F	Thanksgiving Break: No Classes
26	Mon.	Project Presentations
28	Wed.	Project Presentations
December		
3	Mon.	Project Presentations
5	Wed.	Last Day of Class – Wrap Up

Franchetti 18-613: Foundations of Computer Systems, Lecture 4, Based on Material by M. Püschel, ETH Zürich.

<https://courses.ece.cmu.edu/18645>

<http://www.ece.cmu.edu/~franzf/papers/gtfsdcr.pdf>

<https://www.inf.ethz.ch/personal/markusp/teaching/18-645-CMU-spring08/course.html>

[spring08/course.html](https://www.inf.ethz.ch/personal/markusp/teaching/18-645-CMU-spring08/course.html)



Intel® 64 and IA-32 Architectures Software Developer's Manual

Combined Volumes:
1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D and 4

NOTE: This document contains all four volumes of the Intel 64 and IA-32 Architectures Software Developer's Manual: *Basic Architecture*, Order Number 253665; *Instruction Set Reference A-Z*, Order Number 325383; *System Programming Guide*, Order Number 325384; *Model-Specific Registers*, Order Number 335592. Refer to all four volumes when evaluating your design needs.

About 5,000 pages

Order Number: 325462-069US
January 2019



Any Questions?